سync

Multi-Supplier Dropshipping &
Inventory Synchronization Platform

Complete Agent Development Specification

Version 1.0 • Development-Ready • V1 Scope

This document is the primary implementation specification for the coding agent. Development must proceed phase-by-phase, with mandatory QA gates after every phase.

Core workflow

```
SUPPLIERS → ■ync PLATFORM → CENTRAL CATALOG → PRICING + INVENTORY → MARKETPLACES
```

1. PRODUCT OVERVIEW

Build a professional multi-supplier dropshipping and inventory synchronization platform named سync.

The platform acts as a centralized control system between wholesale suppliers and ecommerce marketplaces. Its primary objectives are accurate inventory synchronization, automated pricing and stock updates, overselling prevention, centralized product management, clear error reporting, scalability, and simple operation.

```
SUPPLIERS
■■■ D&H
■■■ Ingram Micro
■■■ TD SYNNEX
■■■ Ma Labs
■■■ VoiceComm
↓
■ync Platform
↓
Catalog + Pricing + Inventory
↓
Amazon / eBay / Walmart / Shopify / Newegg
```

2. CRITICAL DEVELOPMENT RULES

- Build incrementally. Do not implement the entire system at once.
- Every phase must include development, unit tests, integration tests, manual QA, bug fixing, and a final QA gate.
- The agent must not proceed to the next phase until the current phase QA gate passes.
- Real supplier and marketplace credentials are not required to begin development.
- Use clearly separated mock supplier and mock marketplace adapters until real credentials are available.
- The mock adapters must be replaceable by real adapters without changing the core synchronization engine.
- Do not create UI-only or fake functionality. Every UI action must connect to a real backend operation.
- Every backend operation must include validation, error handling, logging, and tests.

3. V1 SCOPE

Supplier functionality

- Supplier registry and connection configuration.
- Product catalog synchronization.
- Inventory and supplier price synchronization.
- Supplier availability/status synchronization.
- Scheduled synchronization, logs, and connection health.

Suppliers: D&H Distributing, Ingram Micro, TD SYNNEX, Ma Labs, VoiceComm.

Marketplace functionality

Marketplaces: Amazon, eBay, Walmart, Shopify, Newegg.

- Select products and marketplaces.
- Publish, update, pause/withdraw listings.
- Synchronize marketplace quantity and price.

Inventory

- Automatic inventory updates, safety buffer, OOS handling, change detection, mismatch detection, and reactivation.

Pricing

- Fixed profit, percentage markup, and marketplace-fee + desired-margin rules.

Dashboard and errors

- Dashboard for product, listing, stock, connection, sync, pricing, and error status.
- Track supplier, marketplace, sync, listing, pricing, missing-data, and inventory-mismatch errors.
- Support email and Slack webhook notifications.

4. V1 EXPLICITLY OUT OF SCOPE

- Automatic order routing.
- Purchase order generation.
- Shipment tracking synchronization.
- Multi-warehouse support.
- Multiple seller-account management.
- Profitability analytics.
- Automated repricing.
- Supplier performance analytics.

These are future expansion features and must not silently become part of V1.

5. TECHNOLOGY STACK

Backend: Python 3.12, FastAPI, Pydantic v2, SQLAlchemy, Alembic.

Background processing: Celery, RabbitMQ, Redis, Celery Beat.

Database: PostgreSQL 16. Search: Elasticsearch. Cache/locks: Redis.

Frontend: Next.js, React, Tailwind CSS.

Storage: AWS S3 or Cloudinary for product images.

Deployment: Docker, Docker Compose, Nginx, HTTPS/Let's Encrypt.

Monitoring: Sentry and Celery Flower.

6. INFRASTRUCTURE

```
Services:
backend
frontend
postgres
redis
rabbitmq
elasticsearch
celery_worker
celery_beat
```

Development must be runnable with Docker Compose and reproducible on a clean development machine.

7. ARCHITECTURE

```
Next.js UI
↓
FastAPI Admin API
↓
Catalog / Pricing / Inventory Services
↓
PostgreSQL
↓
Celery + RabbitMQ
↓
Supplier Adapters → Central Catalog → Marketplace Adapters
```

Use a modular adapter-based architecture. Supplier-specific and marketplace-specific API logic must remain isolated from the core synchronization engine.

8. SUPPLIER ADAPTER ARCHITECTURE

```
SupplierAdapter
fetch_catalog()
fetch_inventory(skus)
fetch_price_changes()
```

Implement mock adapters first. Real supplier adapters must inherit the same abstraction. Supplier-specific fields that do not fit the normalized schema may be stored in JSON.

9. NORMALIZED SUPPLIER DATA

Support: SKU/Supplier SKU, UPC, EAN, MPN, title, brand, description, category, images, specifications, supplier cost, quantity, stock status, shipping information, availability status.

10. MARKETPLACE ADAPTER ARCHITECTURE

```
MarketplaceAdapter
create_listing()
update_listing()
update_inventory()
update_price()
withdraw_listing()
get_listing()
```

Marketplace-specific API requirements must be isolated inside the adapter layer.

11. CORE DATA MODEL

Supplier

```
id
name
adapter_class
credentials_encrypted
is_active
last_synced_at
created_at
updated_at
```

Product

```
id
sku
upc
ean
mpn
```

```
title
brand
description
category
images
specs_json
is_enabled
created_at
updated_at
```

SupplierProduct

```
id
product_id
supplier_id
supplier_sku
cost
qty_available
stock_status
availability_status
shipping_info
last_seen_at
created_at
updated_at
```

Marketplace

```
id
name
adapter_class
credentials_encrypted
is_active
created_at
updated_at
```

Listing

```
id
product_id
marketplace_id
external_listing_id
status
selling_price
listed_qty
last_updated_at
created_at
updated_at
```

PricingRule

```
id
product_id nullable
marketplace_id nullable
rule_type
fixed_amount
percentage
marketplace_fee
desired_margin
notes
created_at
updated_at
```

InventoryRule

```
id
product_id nullable
safety_buffer
out_of_stock_action
created_at
updated_at
```

SyncLog / ErrorLog

```
SyncLog: id, product_id, supplier_id, field_changed, old_value, new_value, synced_at
ErrorLog: id, error_type, product_id, marketplace_id, supplier_id, message, retry_count, status,
resolved_at, created_at
```

SupplierPriority

```
id
product_id
supplier_id
priority_rank
selection_rule
created_at
updated_at
```

12. PRODUCT MATCHING

The same physical product can exist under multiple suppliers. Link supplier products to one central Product using identifiers such as UPC, EAN, MPN, and SKU.

```
Product: Logitech Keyboard
■■■ Ingram Micro: $50 / stock 20
■■■ D&H: $52 / stock 50
■■■ TD SYNNEX: $48 / stock 0
```

Do not implement sophisticated AI matching in V1 unless explicitly requested.

13. SUPPLIER SELECTION

When multiple suppliers provide the same product, support configurable selection based on availability, lowest cost, supplier priority rank, and shipping location.

14. INVENTORY ENGINE

```
marketplace_quantity = max(supplier_quantity - safety_buffer, 0)
```

Example: supplier quantity 8 with safety buffer 2 produces marketplace quantity 6. Never publish negative inventory.

15. OUT-OF-STOCK LOGIC

When supplier quantity reaches zero or a product becomes unavailable, apply the configured action: SET_QUANTITY_ZERO, DISABLE_LISTING, or MARK_UNAVAILABLE.

When stock returns, reactivate according to configuration. Log every state transition.

16. PRICING ENGINE

Fixed profit

```
selling_price = cost + fixed_profit
```

Percentage markup

```
selling_price = cost + (cost × markup_percentage)
```

Fees + desired margin

Support marketplace fees and desired profit margin. Rules may be global, per product, or per marketplace. More-specific rules override less-specific rules.

17. LISTING MANAGEMENT

Users select a product and one or more marketplaces, then publish. Each marketplace listing is tracked independently.

```
Product: Logitech Keyboard
■ Amazon
■ eBay
■ Walmart
■ Shopify
■ Newegg
```

18. SYNCHRONIZATION PIPELINE

```
Supplier
↓
Supplier Adapter
↓
Raw Supplier Data
↓
Validation / Normalization
↓
Central Catalog
↓
Detect Changes
↓
Pricing Rules + Inventory Rules
↓
Marketplace Push Queue
↓
Marketplace Adapter
↓
Marketplace Listing
```

19. CHANGE DETECTION

Compare incoming supplier data with stored data. Detect quantity, cost, availability, title, description, images, category, specifications, and shipping-information changes. Relevant stock/price changes must trigger marketplace synchronization.

20. CELERY TASKS & SCHEDULES

supplier_sync

Fetch, validate, normalize, update database, detect changes, log changes, and trigger marketplace updates. Default: every 15 minutes.

marketplace_push

Calculate quantity and price, update marketplace, record result, and retry failures.

out_of_stock

Detect unavailable products, apply OOS action, and reactivate when stock returns. Default: every 10 minutes.

catalog_reconcile

Compare internal listing state with marketplace state. Default: daily at 02:00.

alerts

Send important alerts through email and Slack.

21. RETRY & CONCURRENCY

- External failures must use retries with exponential backoff and a maximum retry count.
- All failures must be structured and logged.
- Use Redis distributed locks to prevent concurrent workers from updating the same marketplace listing simultaneously.

22. API DESIGN

```
/api/auth
/api/suppliers
/api/suppliers/{id}
/api/suppliers/{id}/sync
/api/products
/api/products/{id}
/api/products/search
/api/marketplaces
/api/listings
/api/listings/{id}
/api/pricing-rules
/api/inventory-rules
/api/sync-logs
/api/errors
/api/dashboard
```

Use Pydantic schemas, validation, pagination, filtering, sorting, and consistent error responses. All list endpoints must be paginated.

23. AUTHENTICATION & SECURITY

- Use JWT authentication for the admin API.
- Never hardcode credentials.
- Use environment variables/secrets management.
- Encrypt supplier and marketplace credentials at rest.
- Enforce HTTPS in production.
- Validate all external API responses.
- Never expose secrets in API responses or logs.

24. DASHBOARD

Required KPI cards: Total Products, Active Listings, In Stock, Out of Stock, Needs Attention.

Show supplier health, marketplace health, last/next sync, recent errors, and products requiring attention.

25. CATALOG & PRODUCT UI

Catalog search: SKU, UPC, EAN, title, brand, MPN. Filters: supplier, marketplace, category, brand, stock, price, listing status, sync status.

Product details must show basic information, images, description, specifications, supplier sources, cost, inventory, marketplace listings, selling price, rules, sync history, and errors.

Use server-side filtering, pagination, database indexes, and Elasticsearch for large catalogs.

26. SUPPLIER & MARKETPLACE UI

Supplier page: connection status, last sync, next sync, imported products, last error, sync history. Actions: connect, disconnect, test connection, sync now, enable/disable, configure schedule.

Marketplace page: connection status, account information where available, active listings, errors, last synchronization. Actions: connect, disconnect, test connection, sync now.

27. ERROR MANAGEMENT

Statuses: SUCCESSFULLY_SYNCHRONIZED, PENDING, LISTING_ERROR, SUPPLIER_CONNECTION_ERROR, MARKETPLACE_CONNECTION_ERROR, MISSING_DATA, PRICING_ERROR, INVENTORY_MISMATCH.

Errors must show enough information for staff to act without opening products individually: timestamp, SKU, supplier/marketplace, message, retry count, and status.

28. MOCK ENVIRONMENT

Create mock versions of all five suppliers and all five marketplaces. The mock environment must simulate product creation, stock changes, price changes, OOS, stock restoration, API failures, slow responses, and authentication failures.

Do not label mock integrations as real integrations in the UI or documentation.

29. REQUIRED END-TO-END TEST

```
Initial:
SKU TEST-001
Cost $500
Quantity 20
Markup 15%
Safety buffer 2

Expected selling price: $575
Expected marketplace quantity: 18

Change cost to $520 and quantity to 8
Expected price: $598
Expected quantity: 6

Change quantity to 0
Expected marketplace quantity: 0

Change quantity to 10
Expected marketplace quantity: 8
```

This scenario must pass before V1 is considered functional.

30. PHASE 1 — FOUNDATION

Create repository structure, FastAPI backend, Next.js frontend, Docker Compose, PostgreSQL, Redis, RabbitMQ, Celery, Celery Beat, database models, Alembic migrations, authentication,

MockSupplierAdapter, catalog APIs, catalog UI, unit tests, and integration tests.

Phase 1 QA Gate

- All infrastructure services start successfully.
- API health, database migrations, CRUD, validation, and authentication tests pass.
- Catalog create/update/delete/search/filter/pagination work.
- Mock catalog, inventory, and price imports work.
- End-to-end Mock Supplier → FastAPI → PostgreSQL → Catalog → Frontend works.

Do not start Phase 2 until all Phase 1 QA checks pass.

31. PHASE 2 — MARKETPLACE FOUNDATION

Implement marketplace abstraction and a complete mock eBay lifecycle. Prepare real eBay Sell API integration architecture.

Support create inventory item, create offer, publish offer, quantity update, price update, and withdrawal conceptually through the adapter.

Phase 2 QA Gate

- Product can be selected and published to mock eBay.
- Supplier stock changes propagate through Celery to marketplace quantity.
- Listing creation/update/withdrawal works.
- Retry, failure logging, and duplicate-update prevention work.

No Phase 3 until Phase 2 QA passes.

32. PHASE 3 — PRICING, INVENTORY & MULTI-SUPPLIER ENGINE

Implement fixed profit, percentage markup, fees + margin, safety buffer, zero floor, OOS, reactivation, multi-supplier linking, supplier priority, and configuration UI.

Phase 3 QA Gate

- Pricing calculations pass.
- Inventory never becomes negative.
- OOS and stock restoration work.
- Multi-supplier selection works for availability, cost, priority, and shipping rules.
- All Phase 1 and Phase 2 regression tests still pass.

33. PHASE 4 — REMAINING INTEGRATIONS

Implement supplier adapters for D&H, TD SYNNEX, Ma Labs, and VoiceComm, plus marketplace adapters for Amazon, Walmart, Shopify, and Newegg. Use mocks when real credentials are unavailable.

Phase 4 QA Gate

Every supplier must pass connection/authentication, catalog, inventory, price, failure, retry, and logging tests. Every marketplace must pass authentication, listing, inventory, price, withdrawal, failure, retry, and logging tests. Run the standardized adapter test suite against every integration.

34. PHASE 5 — DASHBOARD, SEARCH & PRODUCTION

Complete the operational dashboard, Elasticsearch catalog search, Sentry, Celery Flower, structured logging, Nginx, HTTPS, Let's Encrypt, Docker deployment, environment secrets, and database backups.

Phase 5 QA Gate

- Every UI screen is manually tested.
- All APIs are tested.
- All suppliers and marketplaces are integration-tested.
- Stock, price, OOS, restoration, and error scenarios pass.
- Supplier unavailable, marketplace unavailable, timeout, invalid response, invalid credentials, rate limiting, malformed product, missing UPC, missing price, and missing quantity scenarios are handled without crashing.

35. FINAL SYSTEM QA

Run a full regression suite after all phases.

```
Scenario 1: New product → Catalog → Rules → Listing
Scenario 2: Stock 25 → 8 with buffer 2 ⇒ marketplace 6
Scenario 3: Cost $500 → $520 at 15% ⇒ $575 → $598
Scenario 4: Supplier 0 ⇒ marketplace 0 / configured OOS action
Scenario 5: Supplier 10 with buffer 2 ⇒ marketplace 8
Scenario 6: Multiple suppliers ⇒ correct configured supplier
Scenario 7: Marketplace failure ⇒ retry → error log → dashboard → alert
Scenario 8: Concurrent workers ⇒ no conflicting simultaneous update
```

The system must remain operational when individual suppliers or marketplaces temporarily fail.

36. PERFORMANCE

Design for hundreds of thousands of supplier products and thousands of active marketplace listings.

- Use PostgreSQL indexes.
- Use Elasticsearch for large-scale search/filtering.
- Use pagination and server-side filtering.
- Supplier changes should be detected within the configured sync cycle and trigger marketplace updates through the task queue.

37. OBSERVABILITY

Every important operation should include timestamp, operation/task ID, supplier, marketplace, product/SKU, status, duration, error information, and retry count.

38. CODE QUALITY

- Use type hints and Pydantic schemas.
- Keep integrations modular and isolated.
- Avoid duplicated code.
- Use meaningful names.
- Write tests alongside features.
- Use migrations instead of manual production database changes.
- Keep configuration outside source code.

- Do not create giant monolithic service files.

39. TESTING REQUIREMENTS

Unit tests: pricing, inventory, supplier selection, validation, and change detection.

Integration tests: FastAPI + PostgreSQL, Celery + RabbitMQ, Redis locks, supplier adapters, and marketplace adapters.

End-to-end tests: Supplier → Catalog → Rules → Listing → Inventory update.

40. DEFINITION OF DONE FOR EVERY PHASE

```
Implementation complete
↓
Unit tests pass
↓
Integration tests pass
↓
Manual QA complete
↓
Known bugs fixed
↓
Regression tests pass
↓
Documentation updated
↓
Phase QA approved
```

A phase is not complete merely because code has been written.

41. GIT WORKFLOW

Use Git with main, develop, feature/*, and fix/* branches as appropriate.

Milestone commits should include: phase-1-foundation-complete, phase-2-marketplace-complete, phase-3-rules-engine-complete, phase-4-integrations-complete, phase-5-production-complete.

Never commit .env files, API keys, passwords, tokens, supplier credentials, or marketplace credentials.

42. ENVIRONMENT CONFIGURATION

Create .env.example. Include placeholders for DATABASE_URL, REDIS_URL, RABBITMQ_URL, JWT_SECRET, storage credentials, SENTRY_DSN, SLACK_WEBHOOK, supplier credentials, and marketplace credentials.

43. DOCUMENTATION

Maintain README.md, ARCHITECTURE.md, API.md, SETUP.md, TESTING.md, and ENVIRONMENT.md. README must explain how to run the entire system locally.

44. REAL API CREDENTIAL REQUIREMENTS

Development must not be blocked by real credentials.

Eventually request supplier API/feed documentation, credentials, authentication details, catalog/inventory/pricing endpoints, rate limits, IP whitelist requirements, FTP/SFTP details where applicable, and test/sandbox access where available.

For marketplaces request access/accounts for Amazon SP-API, eBay Sell API, Walmart Marketplace API, Shopify Admin API, and Newegg Marketplace API. Determine whether the company already owns the required seller/store accounts.

45. FUTURE EXPANSION

- Automatic order routing.
- Purchase order creation.
- Marketplace order synchronization.
- Supplier order submission.
- Shipment tracking and supplier shipping confirmation.
- Multiple warehouses and seller accounts.
- Profitability reporting and marketplace fee analytics.
- Automated repricing.
- Supplier performance monitoring.
- Product restriction management.
- Advanced staff permissions.
- Sales analytics.
- Automated product matching.
- Additional suppliers and marketplaces.

46. FINAL ACCEPTANCE CRITERIA

The complete workflow must work: Supplier Product → Supplier Adapter → Central Catalog → Product Selection → Pricing Rule → Inventory Rule → Marketplace Listing → Supplier Stock Change → Change Detection → Celery Task → Inventory/Price Calculation → Marketplace Update → Sync Log.

When an error occurs: Failure → Retry → Error Log → Dashboard → Email/Slack Alert.

The platform must remain operational even when an individual supplier or marketplace temporarily fails.

47. MOST IMPORTANT AGENT INSTRUCTION

Treat this document as the primary implementation specification. Do not silently expand V1 scope.

If a real third-party API is unavailable, implement a clearly separated mock adapter. Never pretend a mock integration is a real integration.

Every UI button must connect to a real backend operation. Every backend operation must have validation, error handling, logging, and tests.

Start immediately with Phase 1 only

```
1. Repository structure
2. FastAPI backend
3. Next.js frontend
4. Docker Compose
5. PostgreSQL
6. Redis
7. RabbitMQ
8. Celery + Celery Beat
9. Database models + Alembic
```

```
10. Authentication
11. Mock supplier adapter
12. Catalog APIs + UI
13. Unit + integration tests
14. Complete Phase 1 QA
15. Fix every failure
16. Produce Phase 1 completion report
```

Do not start Phase 2 until Phase 1 QA passes.

48. REQUIRED PHASE COMPLETION REPORT

```
PHASE:
STATUS:

Implemented:
- ...

Tests:
- Total:
- Passed:
- Failed:

QA:
- Passed/Failed

Known Issues:
- ...

Files/Modules Added:
- ...

Next Phase:
- ...
```